
Azure Multi-modal Compliance Ingestion Engine

using LangGraph

Project Type: Portfolio / Solo Development

Methodology: Agile (Solo Sprint-based)

Document Type: Full Planning & Development Guide

Technology Stack

Cloud:	Microsoft Azure
Orchestration:	LangGraph
Backend:	Python, FastAPI
AI/LLM:	Azure OpenAI (GPT-4, Embeddings)
Video:	yt-dlp, Azure Video Indexer
Search:	Azure AI Search (Vector DB)
Storage:	Azure Blob Storage
Observability:	LangSmith, Azure App Insights

Contents

1	Project Overview	2
1.1	Purpose	2
1.2	Architecture Summary	2
1.3	Key Design Decisions	2
2	Development Phases	4
3	Project Folder Structure	8
4	Azure Services Checklist	9
5	Production Checklist	9
6	Key Implementation Notes	10
6.1	Video Ingestion (No Local Storage)	10
6.2	SAS URL Expiry Guidance	10
6.3	LangGraph Node Mapping	10
6.4	Portfolio Interview Talking Points	10
7	Agile Workflow (Solo)	11

1. Project Overview

1.1. Purpose

This project builds a production-grade, multi-modal compliance ingestion engine that:

- Ingests YouTube videos via yt-dlp without storing files locally
- Streams video content directly into Azure Blob Storage
- Extracts OCR, transcripts, and insights using Azure Video Indexer
- Generates vector embeddings and indexes them in Azure AI Search
- Orchestrates the full pipeline using LangGraph agents
- Exposes a REST API via FastAPI for triggering and querying
- Provides full observability via LangSmith and Azure Application Insights

1.2. Architecture Summary

The system is divided into four primary layers:

Layer	Description
Entry Points	CLI trigger (<code>main.py</code>) and FastAPI server backend
Orchestration	LangGraph RAG workflow, Video Processor, Retrieval Engine, Compliance Auditor
Azure Infra	Blob Storage, Video Indexer, Azure AI Search (Vector DB)
External Intelligence	Azure OpenAI (LLM + Embeddings), App Insights, LangSmith

1.3. Key Design Decisions

No Local Storage Policy

Videos are never written to disk. yt-dlp extracts the CDN stream URL, and the stream is uploaded directly to Azure Blob Storage in 4 MB chunks. This ensures the system is stateless and scalable across multiple compute instances.

- **Blob Storage as staging:** Temporary SAS URLs (1–2 hour expiry) are passed to Video Indexer
- **Auto-cleanup:** Blobs are deleted post-indexing; lifecycle policy deletes any orphaned blobs after 24 hours
- **Streaming ingestion:** Only 4 MB lives in RAM at any point during upload

- **LangGraph for orchestration:** Each processing step is a graph node, making the pipeline inspectable and extensible

2. Development Phases

The project is divided into **4 Agile Phases**. Each phase has a clear goal, defined tasks, and a deliverable. Phases are completed sequentially — do not begin the next phase until the current phase deliverable is working end-to-end.

Phase	Name	Goal	Est. Duration
1	Foundation & Ingestion	Video downloaded and indexed in Azure	~2 weeks
2	RAG Pipeline	Insights extracted and semantically searchable	~2 weeks
3	Full LangGraph pipeline wired end-to-end	Compliance & Orchestration	~2 weeks
4	API, Polish & Deployment	Production-ready, deployed, and portfolio-presentable	~2 weeks

Phase 1 Foundation & Ingestion

Goal: Given a YouTube URL, the video is downloaded, streamed to Azure Blob, and indexed by Azure Video Indexer.

Estimated Duration: ~2 weeks (may vary)

Planning Tasks

- Define folder/module structure for the project
- Provision Azure services: Blob Storage, Video Indexer, App Insights
- Set up environment variables and secrets management (`.env` / Key Vault)
- Create GitHub repository with branch strategy (`main`, `dev`, feature branches)
- Set up virtual environment and `requirements.txt`

Development Tasks

1. **yt-dlp stream extraction:** Extract direct CDN URL without downloading locally
2. **Azure Blob upload:** Stream video in 4 MB chunks directly into `temp-videos` container

3. **SAS URL generation:** Generate time-limited SAS URL (1–2 hour expiry)
4. **Video Indexer submission:** Submit SAS URL to Azure Video Indexer API
5. **Polling loop:** Poll Video Indexer job status until complete or failed
6. **Blob cleanup:** Delete blob after indexing completes
7. **Basic logging:** Log key events to console / App Insights
8. **Error handling:** Catch and log exceptions at each step

Deliverable

Running `python main.py -url <youtube_url>` successfully indexes the video in Azure Video Indexer and cleans up the blob.

Phase 2 RAG Pipeline

Goal: Extract insights from the indexed video, generate embeddings, store in Azure AI Search, and enable semantic querying.

Estimated Duration: ~2 weeks (may vary)

Planning Tasks

- Provision Azure AI Search instance and define index schema
- Decide on chunking strategy (fixed-size vs. semantic sentence chunking)
- Choose embedding model (Azure OpenAI `text-embedding-ada-002` or `text-embedding-3-large`)
- Design retrieval query interface

Development Tasks

1. **Transcript extraction:** Pull transcript and OCR output from Video Indexer API
2. **Metadata extraction:** Extract keyframes, topics, named entities, sentiments
3. **Text chunking:** Split transcript into overlapping segments (e.g. 512 tokens, 64 overlap)
4. **Embedding generation:** Call Azure OpenAI embeddings API per chunk
5. **Vector indexing:** Upsert chunks + embeddings into Azure AI Search
6. **Retrieval engine:** Build semantic search function (vector + keyword hybrid)
7. **Query testing:** Manually test queries like *“what did the video say about X?”*
8. **Blob lifecycle policy:** Configure Azure Blob policy to auto-delete after 24 hours as fallback

Deliverable

Given a natural language query, the retrieval engine returns the most relevant transcript segments with timestamps from the indexed video.

Phase 3 Compliance & Orchestration

Goal: Wire all components together using LangGraph, implement the Compliance Auditor agent, and enable end-to-end pipeline execution from CLI.

Estimated Duration: ~2 weeks (may vary)

Planning Tasks

- Design LangGraph node structure (map each diagram box to a node)
- Define graph state schema (what data passes between nodes)
- Define compliance rules / audit criteria for the Compliance Auditor
- Set up LangSmith project for tracing

Development Tasks

1. **Graph state definition:** Define TypedDict or Pydantic model for shared graph state
2. **Node: Ingestion** — wraps Phase 1 video download and indexing logic
3. **Node: RAG Workflow** — wraps Phase 2 embedding and indexing logic
4. **Node: Retrieval Engine** — queries Azure AI Search, returns ranked results
5. **Node: Compliance Auditor** — LLM-based agent that checks content against compliance rules, flags violations
6. **Graph wiring:** Connect nodes with conditional edges (e.g. skip indexing if already indexed)
7. **LangSmith integration:** Enable tracing via `LANGCHAIN_TRACING_V2=true`
8. **CLI trigger:** Wire `main.py` to run the full LangGraph workflow
9. **Azure App Insights:** Log graph execution events and node durations
10. **End-to-end test:** Run full pipeline on 2–3 real YouTube URLs

Deliverable

Running `python main.py -url <youtube_url> -query "..."` executes the full LangGraph pipeline end-to-end, with traces visible in LangSmith and logs in App Insights.

Phase 4 🚀 API, Polish & Deployment

Goal: Expose the pipeline via FastAPI, harden for production, deploy to Azure, and make the project portfolio-presentable.

Estimated Duration: ~2 weeks (may vary)

Planning Tasks

- Design API contract (endpoints, request/response schemas)
- Choose deployment target (Azure App Service, Azure Container Apps, or ACI)
- Plan Docker containerisation strategy
- Plan README and demo materials

Development Tasks

1. **FastAPI setup:** Create `app/` module with router structure
2. **POST /ingest:** Accept YouTube URL, trigger pipeline, return job ID
3. **GET /status/{job_id}:** Return current pipeline status
4. **POST /query:** Accept natural language query, return retrieved results
5. **Input validation:** Use Pydantic models for all request bodies
6. **Error responses:** Consistent error schema (4xx/5xx) with detail messages
7. **Retry logic:** Implement exponential backoff for Video Indexer polling and Azure calls
8. **Dockerisation:** Write `Dockerfile` and `docker-compose.yml`
9. **Azure Key Vault:** Move all secrets from `.env` to Key Vault with Managed Identity
10. **Deploy:** Push to Azure App Service or Container Apps
11. **README:** Architecture diagram, setup guide, example API calls, demo GIF/video
12. **Code quality:** Add type hints, docstrings, `ruff` linting, `pre-commit` hooks

Deliverable

A live, deployed REST API that can ingest a YouTube video, index it, and answer compliance queries — documented and presentable as a portfolio project.

3. Project Folder Structure

azure-compliance-engine/

```
main.py                # CLI entry point
requirements.txt
Dockerfile
docker-compose.yml
.env.example

app/                   # FastAPI application
  __init__.py
  main.py              # FastAPI app instance
  routers/
    ingest.py          # POST /ingest
    query.py           # POST /query
    status.py          # GET /status/{job_id}
  schemas/
    ingest.py
    query.py

pipeline/              # LangGraph pipeline
  graph.py             # Graph definition and wiring
  state.py             # Shared graph state schema
  nodes/
    ingestion.py       # Node: yt-dlp + Blob upload
    rag_workflow.py    # Node: transcript + embeddings
    retrieval.py       # Node: Azure AI Search query
    compliance_auditor.py # Node: LLM compliance check

services/              # Azure service clients
  blob_service.py      # Blob upload, SAS, delete
  video_indexer.py     # Submit, poll, extract
  ai_search.py         # Upsert, query vectors
  openai_service.py    # LLM + embeddings

utils/
  logger.py            # App Insights + console logging
  chunker.py           # Text chunking logic
  config.py            # Settings via pydantic-settings

tests/
  test_ingestion.py
  test_retrieval.py
  test_api.py
```

4. Azure Services Checklist

Service	Purpose	Phase
Azure Blob Storage	Temporary video staging (hot tier, lifecycle policy)	Phase 1
Azure Video Indexer	OCR, transcript, insights extraction	Phase 1
Azure OpenAI	GPT-4 (compliance), text-embedding model	Phase 2
Azure AI Search	Vector DB for semantic retrieval	Phase 2
Azure Application Insights	Logs, metrics, distributed tracing	Phase 3
LangSmith	LangGraph execution tracing and debugging	Phase 3
Azure Key Vault	Secrets management with Managed Identity	Phase 4
Azure App Service / ACA	Hosting the FastAPI backend	Phase 4

5. Production Checklist

Requirement	Phase
No local file storage — stream directly to Blob	1
SAS URLs with short expiry (1–2 hrs)	1
Auto-delete blob after indexing	1
Blob lifecycle policy (auto-delete after 24 hrs as fallback)	2
Retry logic with exponential backoff	4
All secrets in Azure Key Vault (not <code>.env</code>)	4
Structured logging to App Insights	3
LangSmith tracing enabled	3
Input validation on all API endpoints	4
Docker containerised and deployable	4
README with architecture diagram and setup guide	4

6. Key Implementation Notes

6.1. Video Ingestion (No Local Storage)

YouTube URL

```
→ yt-dlp.extract_info(download=False) # Get CDN stream URL
→ requests.get(stream_url, stream=True)
→ container.upload_blob(data=iter_content(chunk_size=4MB))
→ generate_sas_url(expiry=2hrs)
→ video_indexer.submit(sas_url)
→ poll until complete
→ delete_blob()
```

6.2. SAS URL Expiry Guidance

The SAS URL must remain valid for the entire duration of Video Indexer processing.

- Short videos (<10 min): 1 hour expiry is sufficient
- Long videos (>30 min): Set 3–4 hour expiry
- Default recommendation: **2 hours** as a safe baseline

6.3. LangGraph Node Mapping

Each box in the architecture diagram maps to a LangGraph node:

Diagram Component	LangGraph Node
Video Processor (yt-dlp and Indexer)	<code>ingestion_node</code>
RAG Workflow LangGraph	Graph orchestrator
Retrieval Engine	<code>retrieval_node</code>
Compliance Auditor	<code>compliance_node</code>

6.4. Portfolio Interview Talking Points

- **Phase 1:** Demonstrates understanding of cloud ingestion, stateless design, and Azure Blob
- **Phase 2:** Demonstrates understanding of RAG architecture, vector embeddings, and semantic search
- **Phase 3:** Demonstrates understanding of agent orchestration, LangGraph, and observability
- **Phase 4:** Demonstrates ability to ship production-ready, deployed code with clean APIs

Each phase is a **standalone talking point** even if not all phases are complete by interview time.

7. Agile Workflow (Solo)

Even working solo, maintaining lightweight agile practices strengthens the portfolio:

1. **Trello / Notion board:** One column per phase, one card per task
2. **GitHub commits:** Commit after every working feature with descriptive messages
3. **Feature branches:** `feature/phase1-blob-upload`, `feature/phase2-embeddings`, etc.
4. **Pull Requests to main:** Even solo PRs show disciplined workflow to reviewers
5. **Phase tags:** Tag GitHub commits at end of each phase (`v0.1-phase1`, etc.)
6. **README updates:** Update README at end of each phase — shows iterative documentation

Azure Multi-modal Compliance Ingestion Engine — Solo Portfolio Project

All estimated durations are indicative and may vary based on complexity and learning curve.